

Xitique — Project Plan & Technical Specification

Author: @Menete

Date: October 06, 2025

This document contains a complete development plan, technical architecture, security strategy, PostgreSQL data model, ER diagram, roadmap, priorities, requirements and features for the Xitique application. The content is written in English and prepared for a .NET backend implementation with PostgreSQL.

1. Introduction

Xitique is a community savings collection system from Chókwè, Mozambique. This document defines a practical, implementable development plan for the Xitique platform, including architecture, database model (PostgreSQL), security and authentication strategies, development roadmap, and operational requirements.

2. Scope

Build a production-ready RESTful API in .NET, backed by PostgreSQL, providing endpoints for collectors and their customers (ticantes).
Optional web-based admin dashboard and future mobile client in Flutter.

3. Requirements

Functional: Register ticantes, collectors; record daily deposits and debts; compute monthly statements; export reports; auth and RBAC; audit logs; offline sync support for mobile (future).

Non-functional: Security (TLS, encryption), Availability 99.5%, Performance for scaling, Maintainability to evolve to Clean Architecture, Localization (EN + PT).

4. Architecture Overview

Hybrid architecture: presentation (API), application (use-cases), domain (entities), infrastructure (data access). This structure trades initial development speed for future evolvability to Clean Architecture.

Folder Structure Example

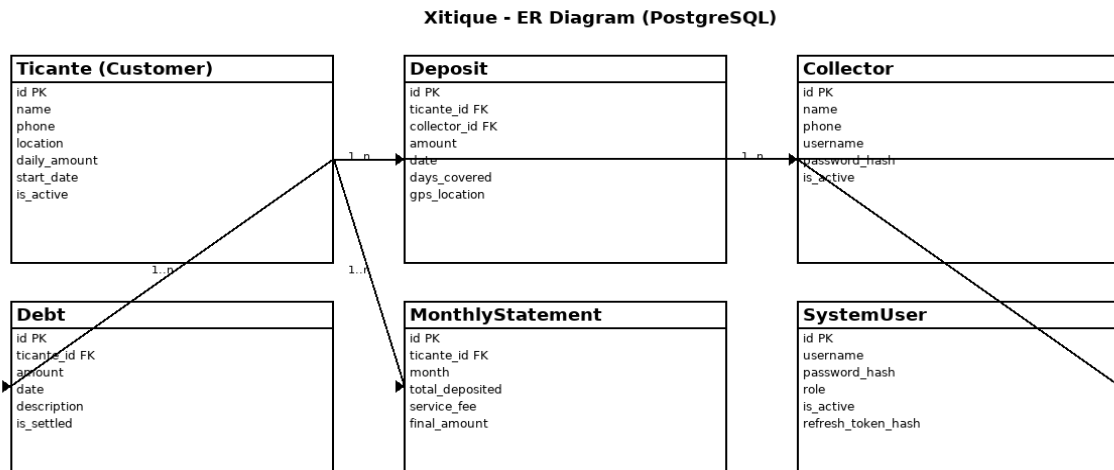
```
/src /Xitique.Api (Controllers, DTOs) /Xitique.Application (Services, Use-cases) /Xitique.Domain (Entities, ValueObjects)
/Xitique.Infrastructure (EF Core, Repositories) /Xitique.Identity (auth, JWT) /tests (unit/integration) /docs /deploy
```

5. Data Model (PostgreSQL)

Main tables: ticante, deposit, collector, debt, monthly_statement, system_user. Use integer PKs (bigserial) or UUIDs if required. Add indexes on ticante_id, collector_id, date.

ER Diagram

Visual Entity-Relationship Diagram for the main data model (PostgreSQL).



6. Authentication and Security

Use JWT access tokens (short-lived) plus refresh tokens. Store refresh token hashes server-side. Passwords hashed with bcrypt or Argon2.

Enforce HTTPS, input validation, rate limiting, and audit logs. Use Serilog + Application Insights for logging and monitoring. Implement DB

backups and encrypted storage for sensitive fields where necessary.

7. Features and API Endpoints

Core endpoints: /auth/login, /auth/refresh, /ticantes (CRUD), /collectors (CRUD), /deposits (create/list), /debts (create/settle),

/statements (generate/export), /reports (daily/monthly).

8. Roadmap (6 months)

Month 0 (2w): Preparation, backlog, infra setup. Month 1 (4w): Core API, auth, ticante & collector CRUD. Month 2 (4w): Deposits & debts

implementation, tests. Month 3 (4w): Monthly statements & reporting. Month 4 (4w): Web dashboard (optional).

Month 5 (4w): Mobile client

groundwork & offline strategy. Month 6 (4w): Testing, production deployment, training.

9. Priorities (MVP)

MVP: auth & user management; ticante & collector CRUD; deposit recording; monthly statement calculation and export; audit logging.

10. Testing Strategy

Unit tests, integration tests, E2E tests for critical flows, security scanning, and load testing for expected peak usage.

11. DevOps & Deployment

CI with GitHub Actions, Docker for containerization, EF Core migrations, deploy to Azure or AWS, automated backups, and health checks.

12. Risks & Mitigations

Delays in requirements -> frequent stakeholder reviews. Collector connectivity issues -> offline support. Data loss -> backups and reconciliation. Security incidents -> dependency scanning and incident response plan.

13. Database DDL Recommendations

Use BIGSERIAL for PKs or UUID, set appropriate FK constraints, tune indexes on frequent queries, and keep financial operations in DB transactions.

14. Appendices: Example DTOs

TicanteDTO { id, name, phone, location, dailyAmount, startDate, isActive } DepositDTO { id, ticanteld, collectorId, amount, date, daysCovered, gpsLocation } MonthlyStatementDTO { id, ticanteld, month, totalDeposited, serviceFee, finalAmount }

Conclusion

This document provides a complete, practical foundation to implement the Xitique project using a .NET backend and PostgreSQL. The hybrid architecture allows quick delivery while enabling future migration to a pure Clean Architecture. Use this document as the single source-of-truth for development, deployment, and handover.